

Diseños experimentales y cuasiexperimentales

Ignacio Spiousas Ismael Calandri

Índice

Prefacio

Este libro *online* surge con la intención de acompañar a los estudiantes de la materia Diseños Experimentales y Cuasiexperimentales de la Licenciatura en Ciencias del Comportamiento de la [Universidad de San Andrés](#) (Victoria, Argentina). Sin embargo, no contiene todo los materiales de la materia, sino que es más bien una guía que nos permite comprender conceptos básicos e identificar lo importante en cada tema. De forma complementaria, en cada capítulo se brindará una lista de bibliografía recomendada.

Lo que vas a aprender en este libro

El objetivo de este libro es introducir a los lectores y lectoras a los conceptos básicos del diseño experimental y cuasiexperimental para que sean capaces de diseñar sus propios experimentos para poner a prueba hipótesis científicas. Asimismo, se busca generar intuiciones que les permitan evaluar diseños experimentales con los que se puedan encontrar tanto en la literatura científica como en cualquier dato obtenido experimentalmente que se les presente al momento de tomar una decisión.

En particular, gran parte de la discusión tendrá que ver con **cómo identificar los efectos causales de un tratamiento**¹ en alguna variable observada. Se comenzará por el tan estimado experimento aleatorizado para luego entrar en las tumultuosas aguas de los diseños cuasiexperimentales.

Se espera que los lectores y lectoras sean capaces de comunicar los diseños experimentales, sus resultados y las implicancias de estos de manera clara, concisa y “apta para todo público”. Asimismo, un concepto transversal es que no siempre es posible implementar un diseño experimental “ideal” (si este existiera)

¹ La palabra *tratamiento* no se refiere únicamente a un tratamiento, por ejemplo, médico, sino que más bien es un término que se utiliza frecuentemente en el diseño experimental, sobre todo en medicina, para referirnos a la causa.

y que tomar decisiones informadas sobre estos (modelos estadísticos, métricas, diseños, etc.) es una parte importante de nuestra labor como generadores de evidencia o tomadores de decisión basada en evidencia.

SIGA SIGA...

Si bien los ejemplos que se presentan en el libro están orientados a las ciencias del comportamiento, su contenido puede ser adaptado a cualquier ciencia experimental, desde la biología hasta la economía.

¿Qué es este libro?

En este libro no vas a encontrar grandes desarrollos teóricos ni ejemplos complejos. Es simplemente una guía de lectura que tiene como objetivo ser la puerta de entrada para algunos conceptos clave del diseño experimental y la inferencia causal. Para esto vamos a chapotear en **R** y *tidyverse*, mojar las patas en la estadística, aguantar un poco la respiración en inferencia y *potential outcomes*, bucear en diseños aleatorizados (experimentos), para finalmente sumergirnos (con la idea de volver a salir a flote) en las profundas aguas del diseño cuasiexperimental y la inferencia causal.

Lo que NO vas a aprender en este libro

Este libro no es un libro de inferencia estadística y no vamos a hablar de *tests* estadísticos, más que en el contexto de una hipótesis científica y de un diseño experimental en particular. Es decir, este libro **NO** es un manual de estadística. Existen cientos de libros que desarrollan en detalle esos contenidos y para nada este libro pretende cubrir esos contenidos.

Este libro tampoco es un manual de programación ni de análisis de datos en **R**. Si bien en la primera sección del libro haremos una presentación de algunas de las funcionalidades de la colección de paquetes para análisis de datos con *tidyverse*, no vamos

a repasar ninguna de las bases de **R** ni de **Rstudio**. Existen infinidad de recursos increíbles para aprender esto y sentimos que no hay necesidad de sumarle a eso uno mediocre.

Prerequisitos

Para sacarle el jugo a los contenidos de este libro hace falta tener conocimientos básicos de probabilidad y estadística, así como estar familiarizado con la programación en R. Ninguno de estos son obstáculos hoy en día ya que hay cientos de fuentes (libros, cursos, etc.) a las que el lector puede consultar previo o durante la lectura de este libro.

DON'T PANIC

El libro comienza con un breve repaso de conceptos básicos de probabilidad y estadística, y presenta los comandos básicos para correr códigos en **R**.

Agradecimientos

Agradecemos profundamente a **Magdalena Cobb, Mora Dal Lago Carabajal, Gabriel Alejandro Díaz, Juan Cruz Menutti, Teresa Laszeski, Gabriela Belén Sanchez y Francisco Strejilevich**, tutores y alumnos asistentes de las clases 2025 y 2026, por su colaboración en la revisión de los contenidos y la edición del libro. También a los alumnos de la clase 2024 y 2025 por sus comentarios y sugerencias durante el desarrollo del libro.

Contacto

Ante cualquier duda pueden contactarme vía e-mail a ispiouas@udesa.edu.ar.

Licencia

Este sitio web es y siempre será gratuito, licencia bajo [CC BY-NC-ND 3.0 License]{<https://creativecommons.org/licenses/by-nc-nd/3.0/us/>}.

1 R y el tidyverse

En la gran mayoría de los ejemplos y ejercicios de este libro vamos a usar una computadora (te quiero mucho Skynet¹). Con ella nos vamos a comunicar utilizando un lenguaje de programación muy popular en el campo de la estadística: **R** (R Core Team 2023). Por eso, mi primera recomendación es que instales **R** y **RStudio**. **RStudio** es una interfaz **muy** popular utilizada para, mayormente, programar en **R**². En el recuadro siguiente van a encontrar información sobre cómo instalar ambas cosas.

💡 Instalación de R y RStudio

Lo primero que hay que hacer para poder correr scripts de R es, como resulta evidente, instalar R. Lo pueden hacer seleccionando su sistema operativo en este [link](#) y siguiendo los pasos de la instalación.

Pueden bajar la versión gratuita de RStudio del siguiente [link](#). En caso de que el link no haya detectado correctamente el sistema operativo, en la sección *All Installers* pueden seleccionarlo manualmente. Una vez descargado el instalador sólo hay que seguir los pasos de instalación.

Si bien la mayoría de las cosas que vamos a hacer en este libro se pueden hacer con funciones de R base³, la propuesta es utilizar los paquetes y funciones del *tidyverse*. El *tidyverse* es una colección de paquetes diseñados para el campo de la ciencia de datos y que comparten una filosofía de diseño subyacente, una gramática y una estructura de datos (Wickham et al. 2019). Tranquilos que en la sección siguiente se va a ir aclarando la cosa.

R Core Team. 2023. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org/>.

² Sin embargo, recientemente *Posit*, la compañía que desarrolla y mantiene RStudio, presentó una nueva interfaz multilenguaje (R, Python y Julia) llamada *Positron* que si, están familiarizados con *Visual Studio Code*, les recomiendo que prueben.

³ Es decir, sin tener que cargar ningún paquete de funciones adicional.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Golemund, et al. 2019. «Welcome to the Tidyverse». *Journal of open source software* 4 (43): 1686.

Unas palabras sobre el uso de modelos de lenguaje (ChatGPT, Claude, Gemini o el que les guste)

No estamos acá para discutir el poder y las capacidades de los grandes modelos de lenguaje (por si no quedó claro, te quiero mucho Skynet⁴). Pero permítannos recomendarles algo: no dejen que la herramienta piense por ustedes antes de que ustedes aprendan a pensar.

El riesgo principal de apoyarse demasiado en la *IA* cuando uno está dando sus primeros pasos en R (o cualquier lenguaje) es lo que los expertos llaman la “ilusión de competencia”. Es esa sensación engañosa de que entendés lo que está pasando porque el código “funciona”, cuando en realidad solo sos un espectador del proceso de resolución. Nuestra sugerencia: usen a los modelos de lenguaje como un tutor, no como un reemplazo. Pregúntenle “¿Por qué este código no funciona?” o “¿Me explicás qué hace esta función de *tidyverse*?”, en lugar de pedirle directamente “Ajustame un modelo lineal con estas variables”.

1.1 Tidy data

Lo primero que tenemos que pensar cuando trabajamos con el *tidyverse* es que nuestros datos estén en formato *tidy*. ¿Qué significa esto? Cuando un *dataset* está en formato *tidy*, cada columna corresponde a una variable y cada fila a una única observación⁴. Veamos un ejemplo. Tenemos tres sujetos a los cuales les medimos el tiempo de respuesta en una tarea. Cada sujeto realiza dos repeticiones de esta medición, el *trial 1* y el *trial 2*. En la tabla Tabla ?? podemos ver las dos formas de organizar esta información.

A lo largo de este capítulo exploraremos los beneficios de almacenar los datos en formato *tidy*. Por supuesto, estas ventajas tienen su precio: el tamaño de las bases de datos crece mucho si tenemos muchas medidas repetidas con distintos valores de las variables.

⁴ El caso contrario sería en el que una fila contiene varias mediciones para distintos niveles de una variable. Este formato se conoce como *wide*.

Tabla 1.1: Ejemplo de tablas tidy y wide.

(a) Tidy			(b) Wide	
sujeto	trial	tiempo_respuesta	trial_1	trial_2
Jerry	1	0.0807501	0.4977774	0.7725215
Jerry	2	0.844330	0.2897672	0.8746007
Elaine	1	0.6007609	0.7328820	0.1749406
Elaine	2	0.1572084		
George	1	0.0073994		
George	2	0.4663935		

1.2 Introducción al Tidyverse

Como contamos más arriba, el *tidyverse* es una colección de cerca de 25 paquetes. Todos ellos están relacionados con la carga, manejo, modificación y visualización de datos. La idea de este libro no es profundizar en todas sus capacidades, pero consideramos importante presentar algunas de las funciones que más vamos a utilizar a lo largo de los capítulos. Estas son funciones para leer datos del paquete *{readr}*, los verbos de *{dplyr}* para manipularlos, las funciones de *{tidyR}* para acomodarlos y el poderosísimo *{ggplot2}* para visualizarlos.

1.2.1 Cargando datos con *readr*

Una de las cosas que vamos a hacer más a menudo en este libro es cargar algún dataset. Para esto vamos a usar varias de las funcionalidades del paquete *{readr}*.

El caso más simple al que nos vamos a enfrentar es la carga de una base de datos organizada en columnas y separadas por comas en un archivo de extensión *.csv*. En este caso, lo que tenemos que hacer es bastante simple: usar la función *read_csv()* como se muestra a continuación.

```
summer <- read_csv("../data/summer.csv")
```

Podemos ver que al cargar los datos `read_csv` nos dice que hay ocho columnas `chr` (o sea, de texto) y una `dbl` (o sea, un número). Si usamos la función `summary` podemos ver un detalle de cada variable con su tipo y alguna descripción⁵:

⁵ Existen alternativas para visualizar rápidamente un conjunto de datos como `str` o `glimpse` o la función `skim` del paquete `{skimr}`.

```
summary(summer)
#>      Year      City      Sport      Discipline
#> Min.   :1896   Length:31165   Length:31165   Length:31165
#> 1st Qu.:1948   Class :character Class :character Class :character
#> Median :1980   Mode  :character Mode  :character Mode  :character
#> Mean   :1970
#> 3rd Qu.:2000
#> Max.   :2012
#>      Athlete      Country      Gender      Event
#> Length:31165   Length:31165   Length:31165   Length:31165
#> Class :character Class :character Class :character Class :character
#> Mode  :character Mode  :character Mode  :character Mode  :character
#>
#>
#>      Medal
#> Length:31165
#> Class :character
#> Mode  :character
#>
#>
#>
```

Los datos que se encuentran dentro de `summer.csv` son los ganadores de medallas en los juegos olímpicos de verano. Podemos ver algunas filas de muestra:

```
head(summer)
#> # A tibble: 6 x 9
#>   Year City Sport Discipline Athlete Country Gender
#>   <dbl> <chr> <chr>    <chr>    <chr>    <chr>    <chr>
#> 1  1896 Athens Aquatics Swimming HAJOS, Alfred HUN Men
#> 2  1896 Athens Aquatics Swimming HERSCHMANN, Otto AUT Men
#> 3  1896 Athens Aquatics Swimming DRIVAS, Dimitrios GRE Men
#> 4  1896 Athens Aquatics Swimming MALOKINIS, Ioannis GRE Men
```

```
#> 5 1896 Athens Aquatics Swimming CHASAPIS, Spiridon GRE Men
#> 6 1896 Athens Aquatics Swimming CHOROPHAS, Efstathios GRE Men
#> # i 2 more variables: Event <chr>, Medal <chr>
```

El formato en el que `read_csv` almacena los datos se llama *tibble* y es el formato por excelencia del *tidyverse*. De momento lo único que nos importa es que es un formato que almacena los casos en filas y las variables en columnas (cada variable tiene un formato). Para más información sobre las cualidades de este formato, les recomendamos revisar la [documentación](#) del paquete *{tibble}*.

1.2.2 El operador *pipe* (`|>`) del paquete *{magrittr}*

El operador pipe nos permite concatenar funciones que utilizan como entrada los mismos datos. El principio de operación es el siguiente: supongan que queremos cargar un dataset y aplicarle la función `summary`. Esto lo podemos hacer simplemente cargando el dataset en una línea de código y ejecutando la función `summary()` de la siguiente forma:

```
data <- read_csv("../data/summer.csv")
summary(data)
#>      Year      City      Sport      Discipline
#> Min.   :1896  Length:31165  Length:31165  Length:31165
#> 1st Qu.:1948  Class :character  Class :character  Class :character
#> Median :1980  Mode  :character  Mode  :character  Mode  :character
#> Mean   :1970
#> 3rd Qu.:2000
#> Max.   :2012
#>   Athlete      Country      Gender      Event
#> Length:31165  Length:31165  Length:31165  Length:31165
#> Class :character  Class :character  Class :character  Class :character
#> Mode  :character  Mode  :character  Mode  :character  Mode  :character
#>
#>
#>
#>      Medal
#> Length:31165
```

```
#> Class :character
#> Mode :character
#>
#>
#>
```

Pero también podemos aprovechar el operador pipe y hacer todo en una única línea de código.

```
read_csv("../data/summer.csv") |> summary()
#>      Year      City      Sport      Discipline
#> Min.   :1896  Length:31165  Length:31165  Length:31165
#> 1st Qu.:1948  Class :character  Class :character  Class :character
#> Median :1980  Mode  :character  Mode  :character  Mode  :character
#> Mean   :1970
#> 3rd Qu.:2000
#> Max.   :2012
#>      Athlete      Country      Gender      Event
#> Length:31165  Length:31165  Length:31165  Length:31165
#> Class :character  Class :character  Class :character  Class :character
#> Mode  :character  Mode  :character  Mode  :character  Mode  :character
#>
#>
#>
#>      Medal
#> Length:31165
#> Class :character
#> Mode  :character
#>
#>
#>
```

Al dejar vacío el paréntesis de la función `summary()`, esta va a tomar como variable de entrada a la que está antes del operador pipe, es decir, a la que antes llamamos `data`. Si la función que se quiere usar tiene más de un argumento, el valor que viene antes del pipe se pasa automáticamente como el primer argumento de la función.

Si bien esta funcionalidad parece algo que complica las cosas y que no trae demasiados beneficios con un ejemplo tan simple,

más adelante veremos que puede ser de gran utilidad, al ayudar a disminuir la cantidad de líneas de código y de variables intermedias.

1.2.3 *{dplyr}* y sus verbos

Una de las cosas más útiles del *tidyverse* -para el tipo de procesamiento de datos que vamos a llevar a cabo en este libro- son los verbos de *{dplyr}*. Estas funciones no permiten agregar columnas, resumir la información, filtrar filas, seleccionar columnas, etc⁶. Todas estas acciones las podemos hacer en la base de datos completa o en una parte de ella agrupada de acuerdo a algún criterio. Vayamos de a poco.

⁶ Para más detalles sobre los verbos disponibles en el paquete *{dplyr}* pueden visitar la [página de referencia](#).

1.2.3.1 El verbo `filter`

Volvamos a los datos de los JJOO de verano. Supongamos que nos queremos quedar sólo con las medallas de Argentina. Para este tipo de filtrado de filas (o casos, o mediciones) *{dplyr}* tiene un verbo que se llama `filter` y funciona de la siguiente forma⁷:

```
summer |> dplyr::filter(Country == "ARG") |> head(10)
#> # A tibble: 10 x 9
#>   Year City Sport Discipline Athlete Country Gender
#>   <dbl> <chr> <chr>   <chr>      <chr>   <chr>   <chr>
#> 1  1924 Paris Athletics Athletics BRUNETO, Luis ARG Men
#> 2  1924 Paris Boxing Boxing PORZIO, Alfredo ARG Men
#> 3  1924 Paris Boxing Boxing QUARTUCCI, Pedro ARG Men
#> 4  1924 Paris Boxing Boxing COPELLO, Alfredo ARG Men
#> 5  1924 Paris Boxing Boxing MENDEZ, Hector ARG Men
#> 6  1924 Paris Polo Polo KENNY, Arturo ARG Men
#> # i 4 more rows
#> # i 2 more variables: Event <chr>, Medal <chr>
```

⁷ Se preguntarán por qué antes de la función `filter` aparece un `dplyr::`. Esto es simplemente una forma de decirle a R que la función `filter` que debe utilizar es la del paquete *{dplyr}*. Esta es una práctica recomendable sobre todo para funciones con nombres comunes como `filter` o `select`.

Noten que estamos utilizando el operador `|>` para concatenar las acciones: con los datos de `summer` hacemos el filtrado y, luego, mostramos las primeras diez filas de esos datos ya filtrados.

También podríamos querer quedarnos con las medallas de Argentina en los JJOO de Atenas 2004, para esto debemos el operador lógico “y”, cuyo símbolo en R es &:

```
summer |> dplyr::filter(Country == "ARG" & Year == 2004) |> head(5)
#> # A tibble: 5 x 9
#>   Year City    Sport    Discipline Athlete          Country Gender
#>   <dbl> <chr>  <chr>      <chr>        <chr>          <chr>  <chr>
#> 1  2004 Athens Aquatics   Swimming    BARDACH, Georgina    ARG    Women
#> 2  2004 Athens Basketball Basketball DELFINO, Carlos Francisco ARG    Men
#> 3  2004 Athens Basketball Basketball FERNANDEZ, Gabriel Diego ARG    Men
#> 4  2004 Athens Basketball Basketball GINOBILO, Emanuel David ARG    Men
#> 5  2004 Athens Basketball Basketball GUTIERREZ, Leonardo Mart~ ARG    Men
#> # i 2 more variables: Event <chr>, Medal <chr>
```

Que linda esa Generación Dorada , ¿no?. Por otro lado, si nos queremos quedar con las medallas de Argentina o Brasil debemos utilizar el operador lógico “o”, cuyo símbolo en R es |:

```
summer |> dplyr::filter(Country == "ARG" | Country == "BRA") |> head(10)
#> # A tibble: 10 x 9
#>   Year City    Sport    Discipline Athlete          Country Gender
#>   <dbl> <chr>  <chr>      <chr>        <chr>          <chr>  <chr>
#> 1  1920 Antwerp Shooting Shooting  PARAENSE, Guilherme    BRA    Men
#> 2  1920 Antwerp Shooting Shooting  BARBOSA, Dario          BRA    Men
#> 3  1920 Antwerp Shooting Shooting  DA COSTA, Afranio Antonio BRA    Men
#> 4  1920 Antwerp Shooting Shooting  PARAENSE, Guilherme    BRA    Men
#> 5  1920 Antwerp Shooting Shooting  SOLEDADE, Fernando      BRA    Men
#> 6  1920 Antwerp Shooting Shooting  WOLF, Sebastiao         BRA    Men
#> # i 4 more rows
#> # i 2 more variables: Event <chr>, Medal <chr>
```

Aunque, cuando queremos filtrar por varios valores de una variable, una alternativa muy útil es agrupar esos valores en un vector utilizando c().

```
summer |> dplyr::filter(Country %in% c("ARG", "BRA")) |> head(10)
#> # A tibble: 10 x 9
#>   Year City    Sport    Discipline Athlete          Country Gender
#>   <dbl> <chr>  <chr>      <chr>        <chr>          <chr>  <chr>
```

```
#> 1 1920 Antwerp Shooting Shooting PARAENSE, Guilherme BRA Men
#> 2 1920 Antwerp Shooting Shooting BARBOSA, Dario BRA Men
#> 3 1920 Antwerp Shooting Shooting DA COSTA, Afranio Antonio BRA Men
#> 4 1920 Antwerp Shooting Shooting PARAENSE, Guilherme BRA Men
#> 5 1920 Antwerp Shooting Shooting SOLEDADE, Fernando BRA Men
#> 6 1920 Antwerp Shooting Shooting WOLF, Sebastiao BRA Men
#> # i 4 more rows
#> # i 2 more variables: Event <chr>, Medal <chr>
```

Finalmente, si tenemos una variable numérica, podemos filtrar con condiciones como mayor o menor, utilizando los símbolos `<` o `>`:

```
summer |> dplyr::filter(Year > 2010) |> head(5)
#> # A tibble: 5 x 9
#>   Year City Sport Discipline Athlete Country Gender
#>   <dbl> <chr> <chr> <chr> <chr> <chr> <chr>
#> 1 2012 London Aquatics Diving BOUDIA, David USA Men
#> 2 2012 London Aquatics Diving QIU, Bo CHN Men
#> 3 2012 London Aquatics Diving DALEY, Thomas GBR Men
#> 4 2012 London Aquatics Diving CHEN, Ruolin CHN Women
#> 5 2012 London Aquatics Diving BROBEN, Brittany AUS Women
#> # i 2 more variables: Event <chr>, Medal <chr>
```

1.2.3.2 El verbo `select`

El verbo `select` es similar a `filter` pero nos permite filtrar variables (o xolumnas) en lugar de casos (o filas). Por ejemplo, ¿qué pasa si sólo nos interesa el año, la ciudad y el nombre del atleta?:

```
summer |> dplyr::select(c(Year, City, Athlete)) |> head(5)
#> # A tibble: 5 x 3
#>   Year City Athlete
#>   <dbl> <chr> <chr>
#> 1 1896 Athens HAJOS, Alfred
#> 2 1896 Athens HERSCHMANN, Otto
#> 3 1896 Athens DRIVAS, Dimitrios
#> 4 1896 Athens MALOKINIS, Ioannis
```

```
#> 5 1896 Athens CHASAPIS, Spiridon
```

1.2.3.3 El verbo mutate

Ahora las cosas se complican un poco. `mutate` es un verbo que nos permite modificar columnas existentes o crear nuevas columnas ya sea con datos nuevos o en función de los datos de nuestra base original. Por ejemplo, con el verbo `mutate` podemos crear una nueva columna llamada `nationality_athlete` que combine el país, guion y el nombre del atleta en una misma variable de tipo *chr*⁸. Luego, podemos quedarnos solo con el año, el tipo de medalla y esta nueva variable⁹.

⁸ Es decir, una cadena de caracteres (aka: un texto).

⁹ Para más detalles sobre la función `paste` pueden ver el siguiente [link](#).

```
summer |>
  dplyr::mutate(nationality_athlete = paste(Country, "-", Athlete)) |>
  dplyr::select(c(Year, Medal, nationality_athlete)) |>
  head(5)
#> # A tibble: 5 x 3
#>   Year Medal nationality_athlete
#>   <dbl> <chr> <chr>
#> 1 1896 Gold  HUN - HAJOS, Alfred
#> 2 1896 Silver AUT - HERSCHMANN, Otto
#> 3 1896 Bronze GRE - DRIVAS, Dimitrios
#> 4 1896 Gold  GRE - MALOKINIS, Ioannis
#> 5 1896 Silver GRE - CHASAPIS, Spiridon
```

O, por ejemplo, podemos querer crear una variable binaria que nos ponga un 1 si es griego y un 0 si no¹⁰:

¹⁰ Para más detalles sobre la función `if_else` pueden ver el siguiente [link](#).

```
summer |>
  dplyr::mutate(is_greek = if_else(Country == "GRE", 1, 0)) |>
  dplyr::select(c(Year, Medal, Country, is_greek)) |>
  head(5)
#> # A tibble: 5 x 4
#>   Year Medal Country is_greek
#>   <dbl> <chr> <chr> <dbl>
#> 1 1896 Gold  HUN      0
#> 2 1896 Silver AUT      0
#> 3 1896 Bronze GRE      1
```

```
#> 4 1896 Gold GRE 1
#> 5 1896 Silver GRE 1
```

Ahora vamos a aprender algo muy importante y *cool* : a agrupar los casos de acuerdo a una variable. Por ejemplo, si queremos agregar una columna que contenga la cantidad total de medallas ganadas por un país a cada fila de ese país:

```
summer |>
  group_by(Country) |>
  dplyr::mutate(num_medals = n()) |>
  dplyr::select(c(Year, Medal, Athlete, num_medals)) |>
  head(5)
#> # A tibble: 5 x 5
#> # Groups:   Country [3]
#>   Country Year Medal Athlete num_medals
#>   <chr>   <dbl> <chr> <chr>      <int>
#> 1 HUN     1896 Gold  HAJOS, Alfred    1079
#> 2 AUT     1896 Silver HERSCHMANN, Otto    146
#> 3 GRE     1896 Bronze DRIVAS, Dimitrios    148
#> 4 GRE     1896 Gold  MALOKINIS, Ioannis    148
#> 5 GRE     1896 Silver CHASAPIS, Spiridon    148
```

¿Perdidos? Tomensé su tiempo para tratar de entender qué pasó y prueben distintas alternativas en sus computadoras.

1.2.3.4 El verbo `summarise`

Por último, el verbo `summarise` nos permite sacar medidas resumen de nuestros datos. Empecemos con algo obvio: ¿Cuántas medallas de oro ganó cada país en la historia de los juegos olímpicos?. Para esto vamos a usar una función de `dplyr` llamada `group_by` que nos permite agrupar los casos de acuerdo a una variable y luego aplicar el verbo `summarise` para obtener, en este caso, la cantidad de medallas de oro por país. Para esto usamos la función `n()` que nos devuelve la cantidad de casos (o filas) que hay en cada grupo y luego la función `arrange` para ordenar los resultados de mayor a menor¹¹.

¹¹ La función `arrange` nos ordena los datos de acuerdo a la variable que le enviemos como parámetro de menor a mayor. Si queremos que ordene de mayor a menor debemos agregar la función `desc` en el argumento. Más detalles [acá](#).

```
summer |>
  dplyr::filter(Medal == "Gold") |>
  group_by(Country) |>
  dplyr::summarise(num_medals = n()) |>
  arrange(desc(num_medals)) |>
  head(10)
#> # A tibble: 10 x 2
#>   Country num_medals
#>   <chr>      <int>
#> 1 USA         2235
#> 2 URS          838
#> 3 GBR          546
#> 4 ITA          476
#> 5 GER          452
#> 6 HUN          412
#> # i 4 more rows
```

Hay algo raro, ¿no? Bueno, sí, de esta forma estamos contando a todos los atletas que tuvieron la misma medalla (por ejemplo, si la medalla fue por fútbol estamos contando cerca de 30 medallas). Para resolver esto nos podemos sacar de encima los casos duplicados por año, deporte, disciplina, evento y género¹²:

```
summer |>
  distinct(Year, Sport, Discipline, Event, Gender,
  dplyr::filter(Medal == "Gold") |>
  group_by(Country) |>
  dplyr::summarise(num_medals = n()) |>
  arrange(desc(num_medals)) |>
  head(5)
#> # A tibble: 5 x 2
#>   Country num_medals
#>   <chr>      <int>
#> 1 USA         67
#> 2 GBR         46
#> 3 CHN         40
#> 4 RUS         22
#> 5 GER         19
```

¹² La función `distinct` elimina filas duplicadas, dejando una sola observación por cada combinación única de las variables que le pasemos como parámetros. Más detalles [aquí](#).

Vayamos con lo último: calculemos la media y la desviación estándar de las medallas de Argentina por JJOO combinando

todo lo que vimos.

```
summer |>
  distinct(Year, Sport, Discipline, Event, Medal, Gender, .keep_all = TRUE) |>
  dplyr::filter(Country == "ARG") |>
  group_by(Country, Year) |>
  dplyr::summarise(num_medals = n()) |>
  ungroup() |>
  summarise(media = mean(num_medals),
            desvio = sd(num_medals))
#> # A tibble: 1 x 2
#>   media desvio
#>   <dbl> <dbl>
#> 1  3.83  2.28
```

Digieran esto tranquilos.

1.2.4 *{tidyR}*, el paquete para ordenar tus datos

El paquete *{tidyR}* tiene muchas herramientas de manejo de tablas como reformatear, expandir tablas, manejar valores faltantes, dividir celdas, anidar datos, etc¹³. Sin embargo, en esta breve introducción sólo vamos a presentar muy brevemente las herramientas que nos permiten convertir una tabla *wide* en *tidy* (o *long*) y viceversa.

¹³ Para más información ver el [cheatsheet](#).

1.2.4.1 La función `pivot_longer`

Volvamos a la tabla inicial que teníamos en formato *wide*:

```
tabla_wide <- tibble(sujeto = rep(c("Jerry", "Elaine", "George")),
                    trial_1 = runif(3),
                    trial_2 = runif(3))

tabla_wide
#> # A tibble: 3 x 3
#>   sujeto trial_1 trial_2
#>   <chr>    <dbl>  <dbl>
#> 1 Jerry    0.320  0.404
```

```
#> 2 Elaine    0.402  0.0637
#> 3 George    0.196  0.389
```

Si nosotros quisiéramos transformar esta tabla en una tabla en formato *tidy* podríamos utilizar la función `pivot_longer`¹⁴. Veamos como funciona y después la desmenuzamos:

¹⁴ Más información [acá](#).

```
pivot_longer(data = tabla_wide,
             cols = trial_1:trial_2,
             names_to = "trial",
             values_to = "tiempo_respuesta")
#> # A tibble: 6 x 3
#>   sujeto trial tiempo_respuesta
#>   <chr>   <chr>           <dbl>
#> 1 Jerry trial_1           0.320
#> 2 Jerry trial_2           0.404
#> 3 Elaine trial_1           0.402
#> 4 Elaine trial_2           0.0637
#> 5 George trial_1           0.196
#> 6 George trial_2           0.389
```

Los argumentos son los siguientes: `data` es la tabla a la que le vamos a realizar el cambio de formato; `cols` son las columnas que vamos a cambiar, en este caso desde `trial_1` a `trial_2`; en `names_to` indicamos la variable a la que vamos a mandar los nombres de las columnas actuales; y `values_to` la variables a la que vamos a mandar los valores.

Algo ligeramente raro es que la columna `trial` no es numérica y, sólo por completitud, lo vamos a solucionar usando a nuestro gran amigo `|>` y al verbo `mutate`¹⁵:

¹⁵ Y la función `parse_number` del paquete `{readr}`.

```
pivot_longer(data = tabla_wide,
             cols = trial_1:trial_2,
             names_to = "trial",
             values_to = "tiempo_respuesta") |>
  mutate(trial = parse_number(trial))
#> # A tibble: 6 x 3
#>   sujeto trial tiempo_respuesta
#>   <chr>   <dbl>           <dbl>
```

```
#> 1 Jerry      1      0.320
#> 2 Jerry      2      0.404
#> 3 Elaine     1      0.402
#> 4 Elaine     2      0.0637
#> 5 George     1      0.196
#> 6 George     2      0.389
```

1.2.4.2 La función `pivot_wider`

Ahora vamos con el caso contrario en el que tenemos una tabla en formato *long* y la queremos convertir en *wide*:

```
tabla_long <- pivot_longer(data = tabla_wide,
                           cols = trial_1:trial_2,
                           names_to = "trial",
                           values_to = "tiempo_respuesta") |>
  mutate(trial = parse_number(trial))

tabla_long
#> # A tibble: 6 x 3
#>   sujeto trial tiempo_respuesta
#>   <chr>   <dbl>           <dbl>
#> 1 Jerry     1      0.320
#> 2 Jerry     2      0.404
#> 3 Elaine    1      0.402
#> 4 Elaine    2      0.0637
#> 5 George    1      0.196
#> 6 George    2      0.389
```

Para esto vamos a echar mano a la función `pivot_wider`¹⁶ que tiene una sintáxis parecida a su prima `pivot_longer`:

¹⁶ Más información [acá](#).

```
pivot_wider(data = tabla_long,
             names_from = trial,
             values_from = tiempo_respuesta)

#> # A tibble: 3 x 3
#>   sujeto `1` `2`
#>   <chr>   <dbl> <dbl>
```

```
#> 1 Jerry 0.320 0.404
#> 2 Elaine 0.402 0.0637
#> 3 George 0.196 0.389
```

Et Voilà!, ya tenemos nuestra tabla en formato wide. En este caso, le dijimos de que variable tomar los nombres de las nuevas columnas en `names_from` y de que variable tomar los valores en `values_from`.

Finalmente, y sólo para alimentar nuestra obsesión, vamos a corregir los nombres de las columnas agregando el prefijo `trial_` utilizando el parámetro de la función `names_prefix`:

```
pivot_wider(data = tabla_long,
            names_from = trial,
            names_prefix = "trial_",
            values_from = tiempo_respuesta)
#> # A tibble: 3 x 3
#>   sujeto trial_1 trial_2
#>   <chr>    <dbl>  <dbl>
#> 1 Jerry    0.320   0.404
#> 2 Elaine    0.402   0.0637
#> 3 George    0.196   0.389
```

1.2.5 *{ggplot2}* o cómo hacer figuras que sean la envidia de tu compañero de escritorio

Tradicionalmente, R incluye funciones gráficas básicas -como `plot()`, `hist()` o `boxplot()`-. Estas funciones generan gráficos simples, pero muy útiles cuando queremos dar una mirada rápida a nuestros datos durante el trabajo. Sin embargo, el paquete *{ggplot2}* es una opción superadora ya que nos permite crear gráficos complejos, es extremadamente flexible y nos permite reutilizar gran parte del código creado. La idea central de *{ggplot2}* es introducir una forma nueva de pensar, definir y programar gráficos: **Grammar of Graphics**.

1.2.5.1 Grammar of Graphics o el idioma de ggplot

El concepto detrás de *{ggplot2}* es que cualquier gráfico puede descomponerse en elementos fundamentales:

- Datos
- *Aesthetics* o mapeos estéticos
- *Geoms* o geometrías
- Escalas
- Facetas
- Temas

Sin embargo, el aspecto más importante para entender este sistema es que estos elementos se agregan de manera modular y en capas, donde se utiliza el símbolo `+` para ir sumando cada nueva capa al gráfico.

Cada elemento se superpone al que sigue en el orden que los escribimos, es decir que, si por ejemplo, en nuestra instrucción primero pedimos un boxplot, y después un gráfico de puntos, vamos a ver los puntos sobre el boxplot. Si los escribiéramos al revés, puede pasar que nuestro boxplot oculte algunos puntos. En otras palabras, un gráfico no es una única instrucción, sino una construcción progresiva, en donde podemos superponer geometrías. Veamos qué queremos decir con esto.

Acá hay un código de ejemplo utilizando el dataset `mtcars` que ya viene con **R** y tiene datos de autos. También podemos ver su gráfica resultante.

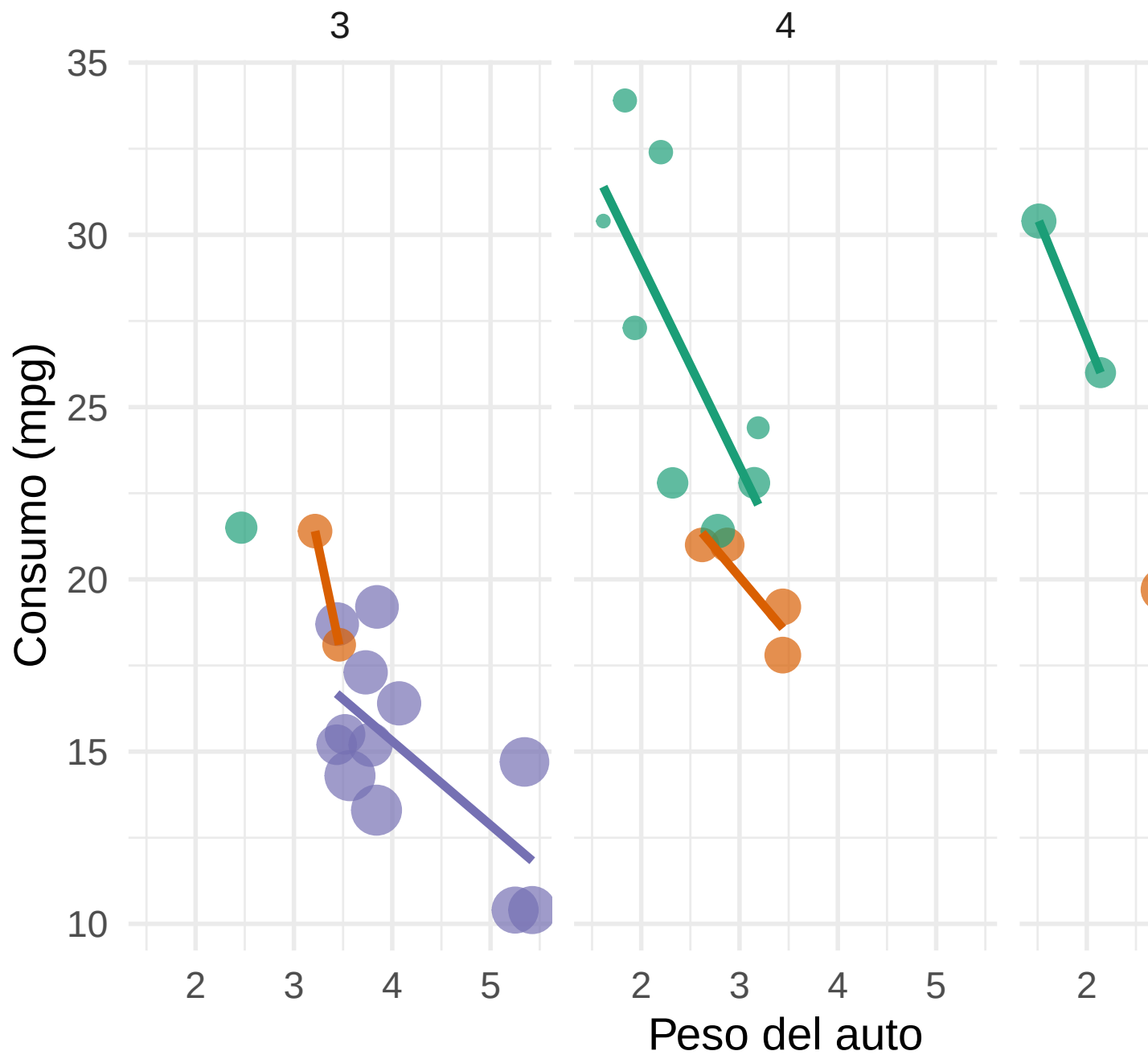
```
ggplot(  
  data = mtcars, # 1. DATOS de autos que ya vienen con R  
  aes(  
    x = wt, # 2. MAPEO ESTÉTICO  
    y = mpg,  
    color = factor(cyl),  
    size = hp  
  )  
) +
```

```

geom_point(alpha = 0.7) + # 3. GEOMETRÍA
geom_smooth(method = "lm", se = FALSE) + # Otra GEOMETRIA COMO capa adicional
scale_color_brewer(palette = "Dark2") + # 4. ESCALAS
facet_wrap(~ gear) + # 5. FACETAS
labs( # Etiquetas de los ejes y las escalas
      title = "Ejemplo de Grammar of Graphics",
      x = "Peso del auto",
      y = "Consumo (mpg)",
      color = "Cilindros",
      size = "Potencia"
) +
theme_minimal() #6. TEMA

```

Ejemplo de Grammar of Graphics



Como ven, este gráfico tiene dos *geometrías* (los puntos y la línea de tendencia), una escala de colores personalizada, *facet* por cantidad de engranajes y un *tema* minimalista. Todo esto se puede lograr con un código relativamente simple y modular. Vamos a descomponer esta estructura para entenderla mejor.

1.2.5.1.1 Datos y mapeo estético

Estos dos componentes son los cimientos de cualquier gráfico de ggplot, sin ellos no puede existir en un gráfico y se delimitan en la primera línea de nuestro gráfico dentro de la función ggplot. **Datos** Todo gráfico comienza con un conjunto de datos. En *{ggplot2}*, estos se especifican con la función `ggplot()` de la siguiente manera:

```
ggplot(data = datos)
#donde datos es el nombre de nuestro dataset
```

Aquí no se genera aún ningún gráfico. Solo se define la base sobre la cual se construirán las capas. Esto refleja un principio clave: *{ggplot}* separa la definición del gráfico de su representación visual.

Por otro lado, los mapeos estéticos (o, en palermitano, *aesthetics*) definen cómo las *variables del dataset* se traducen en propiedades visuales, es decir, cómo usará valores dentro del dataset para graficar algo como:

- Posiciones en el eje X
- Posiciones en el eje Y
- Colores (se puede asignar colores a categorías o escalas de colores a variables continuas)
- Tamaños: podemos tener objetos mas grandes o mas chicos en función de una variable numérica en nuestro dataset
- Formas
- Niveles de transparencia

Estos mapeos se colocan dentro del argumento `aes()` dentro de la función `ggplot()`. Por ejemplo:

```
ggplot(data = datos, aes(x = edad, y = memoria))  
# Lle estamos diciendo que en datos hay una variable edad que queremos mapear al eje x y una
```

En este punto, todavía no hay una representación gráfica (aún estamos en la primera línea). Sólo se establece cómo se relacionan las variables con los ejes.

💡 Mapeo estético vs parámetros fijos

Es importante distinguir entre:

Mapeo estético: cuando el color o tamaño dependen de una variable, se fijan en la primera línea y la información la va a buscar en el dataset, si cambia en un sujeto esa variable, en el gráfico tendrá distinto tamaño o color

```
ggplot(data = datos, aes(x = edad, y = memoria, color=genero))  
# Cuando en el dataset un sujeto tenga -por ejemplo- género masculino tendrá un color, y g
```

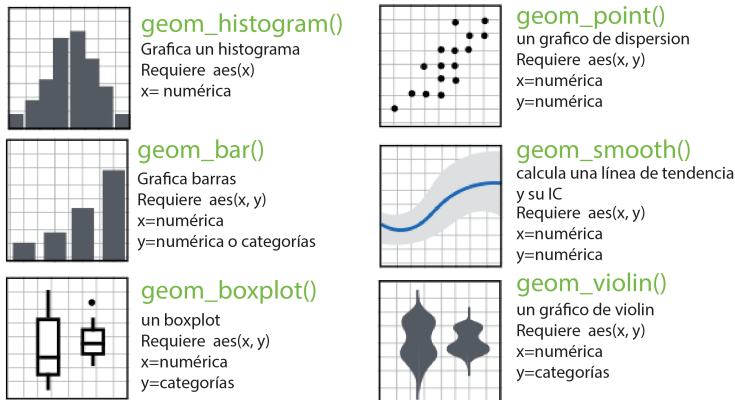
Parámetros fijos: cuando el color o tamaño son constantes, ya no están asociados a una variable simplemente el usuario lo determina, siempre va a ser el mismo.

```
ggplot(data = datos, aes(x = edad, y = memoria, color="#FF0000"))  
# Todo lo que grafiquemos va a tener el mismo color, en este caso rojo.
```

1.2.5.1.2 Geoms o geometría

Las geometrías (*geoms*) son los elementos visuales que representan los datos en el gráfico. Son las capas que se superponen sobre la base definida por los datos y los mapeos estéticos. Hasta que no coloquemos esta capa, no veremos un gráfico (porque la primera línea son los cimientos esenciales, pero que no se ven). Hay muchas geometrías disponibles en `{ggplot2}`, cada una diseñada para representar diferentes tipos de datos y relaciones.

Vamos por ejemplo algunas:



Dos cosas a destacar son que (como dijimos antes) ggplot no va a graficar nada si no le decimos dónde están los datos ni los mapeamos (la primera línea) y si no tiene una geometría. Como ven aquí entonces cada geometría depende de que estén disponibles cierto tipo de datos en el mapeo estético. Por ejemplo no puede crear una línea de tendencia si x e y son categorías.

Hay muchísimas geometrías, los invito a que las exploren y busquen cuáles son las que pueden funcionar mejor para cada uno de sus datos. Un buen lugar para comenzar esta búsqueda es la [galería de gráficos de R](#).

Habrán notado que cada capa de una geometría es una función, cada una con sus propios argumentos, generalmente estos argumentos nos permiten modificar la estética de esa geometría. Por ejemplo, en el caso de `geom_point` podemos modificar el tamaño de los puntos con el argumento `size`, la transparencia con `alpha` o la forma con `shape`. En el caso de `geom_smooth` podemos elegir el método de ajuste (por ejemplo, una línea de tendencia lineal) con el argumento `method` o si queremos mostrar o no el intervalo de confianza con `se`. Cada geometría tiene sus propios argumentos, pero también hay argumentos comunes a todas las geometrías como `color`, `fill`, `size`, etc.

💡 Mapeos en las geometrías

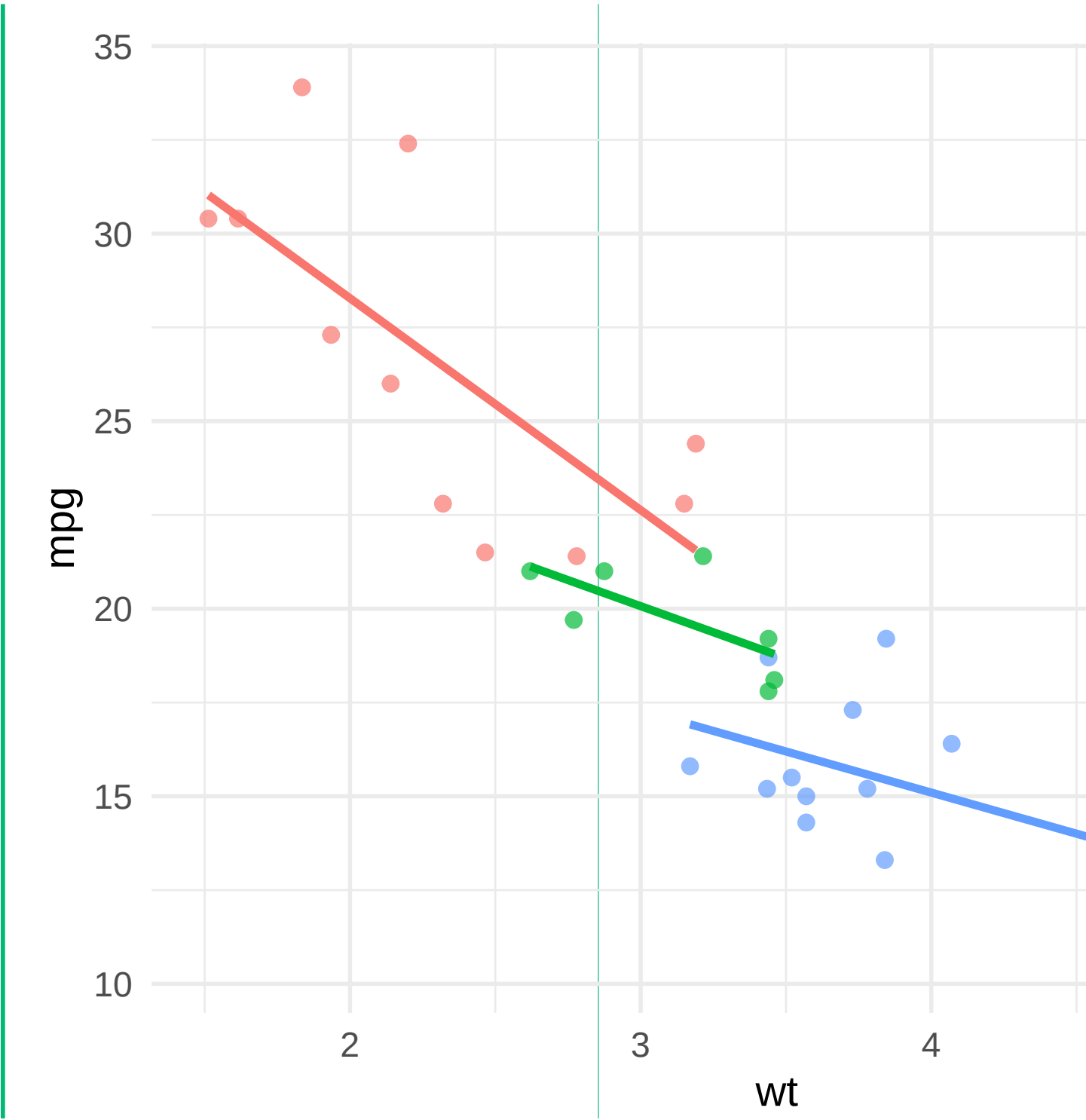
Habrán notado que `size`, `shape` o `alpha` (la transparencia), son argumentos que mencionamos en la capa de

mapeo estético. También dijimos que pueden ser mapeos o parámetros fijos. Bueno, el mapeo estético no es una función exclusiva de la primera capa, puede ser un argumento dentro de una geometría, pero tenemos que saber que **solamente** va a ser usada para esa geometría y no va a afectar a las otras.

Para entenderlo mejor, si el mapeo estético se define en la primera línea del gráfico (`ggplot()`), afecta a todas las geometrías. En cambio, si se define dentro de una geometría específica (`geom_*()`), solo afecta a esa capa.

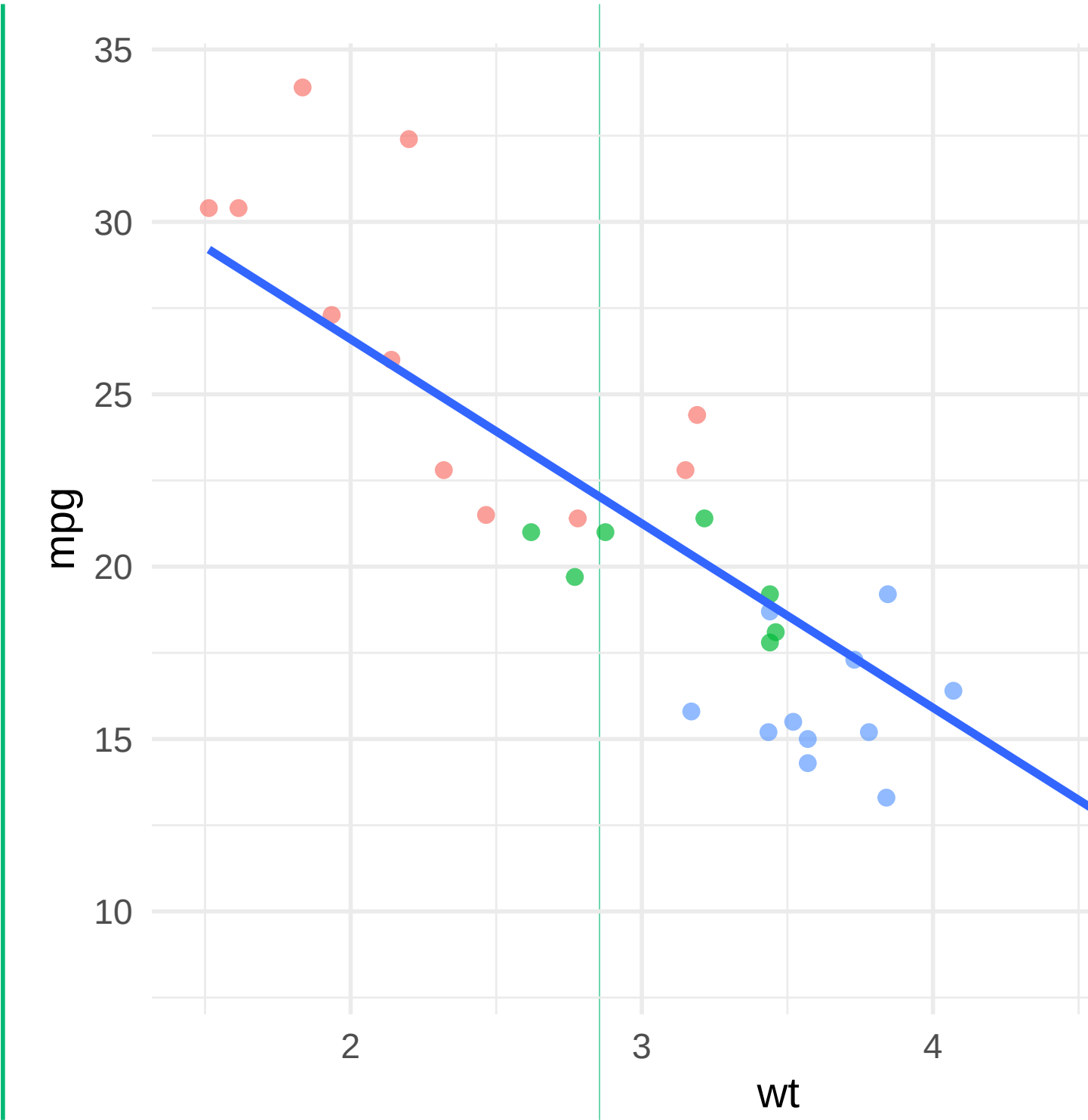
Veamos un ejemplo.

```
ggplot(  
  data = mtcars, # 1. DATOS  
  aes(x = wt, # 2. MAPEO ESTÉTICO  
      y = mpg,  
      color = factor(cyl))) +  
  geom_point(alpha = 0.7) + # 3. GEOMETRÍAS  
  geom_smooth(method = "lm", se = FALSE) +  
  theme_minimal()
```



En este caso, el mapeo estético de `color` está en la primera línea por lo que afecta a la primera capa de geometrías `geom_point` y a la segunda `geom_smooth` creando una línea de tendencia por cada tipo de cilindrada a la que asigno un color

```
library(ggplot2)
ggplot(
  data = mtcars, # 1. DATOS
  aes(x = wt, # 2. MAPEO ESTÉTICO
      y = mpg)) +
  geom_point(alpha = 0.7, aes(color = factor(cyl))) + # 3. GEOMETRÍAS
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal()
```



En esta versión en cambio el mapeo estético de color se encuentra dentro de `geom_point` (afecta solo a esta) dando lugar así a una única línea en `geom_smooth` (porque esa capa no tiene indicado un mapeo de color).

Como ven, hay otros argumentos dentro de la geometría como `size` dentro de `+` que son parámetros fijos del tamaño de los puntos y no están ligados a ninguna variable

1.2.5.1.3 Capas adicionales

Una vez que hemos dominado la estructura de cimientos (data y mapeo estético) y las capas de geometrías estamos en condiciones de generar la gran mayoría de gráficos que necesitamos, pero las posibilidades de *{ggplot2}* son mucho más amplias, hablaremos de capas adicionales que pueden mejorar mucho los resultados o enriquecen la visualización.

Escalas Las escalas controlan cómo los valores se traducen en colores, tamaños o transformaciones.

```
scale_color_continuous()  
# Controla qué colores se usan para representar una variable numérica cuando esta está mapeada
```

También pueden transformar los ejes:

```
scale_x_log10()
```

Esto permite representar relaciones no lineales de forma más clara.

Facetas

Las facetas permiten dividir el gráfico en subgráficos según una variable. Esto facilita comparar grupos.

Por ejemplo:

```
facet_wrap(~ sexo)  
# Divide el gráfico inicial en uno para cada sexo
```

Temas

Los temas controlan la apariencia general:

- Fondo
- Tipografía
- Líneas
- Grilla

Se pueden usar temas predefinidos como `theme_minimal()` o `theme_classic()`, o personalizar cada elemento utilizando la función `theme()`. Esta última permite controlar casi todos los aspectos visuales del gráfico, desde el tamaño y estilo de las fuentes hasta el color de fondo y las líneas de la grilla. Por ejemplo, podemos modificar el título del gráfico, los títulos de los ejes, el color de los valores de los ejes, las líneas principales de la grilla, etc.

Veamos un ejemplo:

```
theme(  
  # Modifica el título del gráfico  
  plot.title = element_text(  
    size = 16,      # Tamaño de la fuente  
    face = "bold",  # Negrita  
    hjust = 0.5     # Centra el título (0 = izquierda, 1 = derecha)  
  ),  
  
  # Cambia el tamaño de los títulos de los ejes  
  axis.title = element_text(  
    size = 12  
  ),  
  
  # Modifica el texto de los ejes (números)  
  axis.text = element_text(  
    color = "darkred" # Color de los valores de los ejes  
  ),  
  
  # Personaliza las líneas principales de la grilla  
  panel.grid.major = element_line(  
    color = "gray80"  # Color suave para mejorar legibilidad  
  ),
```

```

# Elimina las líneas secundarias de la grilla
panel.grid.minor = element_blank(),

# Cambia el fondo del panel donde están los datos
panel.background = element_rect(
  fill = "white"      # Fondo blanco del área del gráfico
),

# Cambia el fondo general del gráfico
plot.background = element_rect(
  fill = "gray95"     # Color de fondo externo
)
)

```

💡 Capas adicionales y la grammar of graphics

No está de más recordar que la *grammar of graphics* rige para todas estas funciones y las reglas de adición son las mismas. Por ejemplo:

```

theme(
  # Modifica el texto de los ejes (números)
  axis.text = element_text(
    color = "darkred"  # Color de los valores de los ejes
  )+
  theme_minimal()

```

En este caso el tema minimalista se va a superponer al tema personalizado en que cambiamos el color del texto del eje, por lo que el resultado final va a ser un gráfico minimalista sin cambio en el color de los valores de los ejes. Si queremos que el tema minimalista sea la base y luego personalizarlo con nuestro tema personalizado, el orden de las capas debe ser el opuesto.

1.2.5.2 Instrucciones finales de ggplot

El paquete *{ggplot2}* es una herramienta muy versátil y poderosa para la visualización de datos. La combinación de su

estructura modular, la amplia variedad de geometrías y la capacidad de personalización lo convierten en una opción ideal para crear gráficos informativos y atractivos. A medida que te familiarices con sus componentes y funciones, vas a poder aprovechar al máximo su potencial para comunicar tus datos y tus *insights* de manera efectiva.

Sólo algunas recomendaciones antes de crear tus propios gráficos. Recordá:

- definí claramente tus datos y mapeos estéticos;
- elegí la geometría adecuada para tu tipo de datos y mensaje;
- Personalizá tu gráfico con escalas, facetas y temas para mejorar la claridad y la estética;
- **no te olvides de la lógica de capas:** el orden en que agregas las capas puede afectar el resultado final. Es fundamental pensar en términos de construcción progresiva;
- **no confundas mapeos y parámetros:** Asegurate de entender la diferencia entre mapeos estéticos (que dependen de las variables) y parámetros fijos (que son constantes). Esto es crucial para evitar confusiones en la interpretación de tus gráficos;
- **asegurate que tus datos estén limpios y el tipo de variable bien definido:** *{ggplot2}* es muy poderoso pero también es sensible a la calidad de los datos. Chequeá que tus datos estén limpios, sin valores faltantes o errores, y que las variables estén correctamente definidas (por ejemplo, factores para categorías). Esto va a facilitar que el mapeo estético y el gráfico final funcionen bien.

Si quieren saber más de *{ggplot2}* les recomiendo [este hermoso tutorial](#) de uno de los gurúes de la visualización de datos con *{ggplot2}*, Cedric Scherer.

1.3 Algunas palabras finales

Como vimos brevemente en este capítulo, los paquetes del *tidyverse* son una herramienta importantísima para el análisis de

datos utilizando **R**. Para más detalles sobre estas funcionalidades les recomendamos la guía de Hadley Wickham (Wickham et al. 2019) o, si ya se quieren sumergir de lleno en el mundo del análisis de datos con R, este fantástico libro [Wickham, Çetinkaya-Rundel, y Grolemund (2023)]¹⁷.

1.4 Preguntas de repaso

1. Tenés datos sobre performance en un juego de tres sujetos: Fran, Lu y Mateo. A cada uno, lo mediste tres veces, y en cada medición, registraste tiempo de reacción, tasa de respuestas correctas correcta y clicks.
 - a) ¿Cómo ordenarías estos datos para que estén en formato *tidy*? (explicar cómo ordenarías las filas y columnas en una tabla)
 - b) ¿Qué ventajas trae almacenar los datos en formato *tidy*? ¿Qué desventajas trae?
 - c) ¿Qué funciones se podrían utilizar para ordenar los datos de formato *wide* a *tidy* y viceversa?
2. ¿Qué significa que un dataset esté en formato *tidy*?
3. ¿Cuál es la diferencia entre el formato *tidy* (o *long*) y el formato *wide*? Mencioná al menos una ventaja y una desventaja de almacenar los datos en formato *tidy*.
4. Explicá qué hace el operador pipe (`|>`) y cuál es su utilidad
5. Describí para qué se usa cada uno de los siguientes verbos de *{dplyr}*: `filter()`, `select()`, `mutate()` y `summarise()`. ¿En qué se diferencia el resultado de usar `mutate()` en combinación con `group_by()` del resultado de usar `summarise()` en combinación con `group_by()`?
6. En el contexto del paquete *{ggplot2}*, ¿qué significa *Grammar of Graphics*? ¿Cuáles son los elementos fundamentales de este sistema?

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Gromund, et al. 2019. «Welcome to the Tidyverse». *Journal of open source software* 4 (43): 1686.

Wickham, Hadley, Mine Çetinkaya-Rundel, y Garrett Gromund. 2023. *R for data science*. "O'Reilly Media, Inc."

¹⁷ Disponible gratis online en [acá](#).

7. Supongamos que hacés un gráfico de dispersión con puntos utilizando funciones de *{ggplot2}* y le agregás una línea de tendencia. Ves que algunos de los puntos que te interesan quedan tapados por la línea. ¿Qué puede estar pasando? ¿Cómo lo corregirías?

2 Repaso de probabilidad y estadística

Hola y bienvenidos a este repaso de probabilidad y estadística. En esta sección vamos a repasar algunos conceptos básicos de probabilidad y estadística que nos van a ser de vital importancia para lo que sigue. No vamos a entrar en detalle¹⁸, ni vamos a hacer demostraciones, sino que vamos a tratar de **entender los conceptos de forma intuitiva y práctica**.

2.1 Eventos aleatorios

Un evento aleatorio es un resultado posible de un experimento aleatorio. Por ejemplo, si tiramos una moneda al aire, los eventos aleatorios posibles son “cara” y “ceca”. Si tiramos dos monedas, los eventos aleatorios posibles son “cara-cara”, “cara-ceca”, “ceca-cara” y “ceca-ceca”. En general, el conjunto de todos los eventos aleatorios posibles se llama espacio de eventos y se denota con la letra Ω .

2.2 Variables aleatorias

Wasserman(Wasserman 2004) nos dice que una variable aleatoria es un mapeo entre el espacio de eventos y los números reales ($X : \Omega \rightarrow \mathbb{R}$). Momento cerebritito ¿Esto que quiere decir? En términos prácticos, lo que implica esta definición es que una variable aleatoria nos da un número para cada evento del posible espacio de eventos aleatorios.

Vamos con un ejemplo que nos aclare un poco las cosas. Supongan que tiramos una moneda justa¹⁹ dos veces y tenemos la variable aleatoria X que cuenta la cantidad de caras

¹⁸ Repetimos la recomendación. Si quieren profundizar en alguno de los temas pueden leer el capítulo 1 de (Wasserman 2004) o el repaso de probabilidad de (Cunningham 2021) (disponible online).

Wasserman, Larry. 2004. *All of statistics: a concise course in statistical inference*. Springer Science & Business Media.

Cunningham, Scott. 2021. *Causal inference: The mixtape*. Yale university press.

Wasserman, Larry. 2004. *All of statistics: a concise course in statistical inference*. Springer Science & Business Media.

¹⁹ Una moneda justa es aquella que tiene la misma probabilidad de dar cara o ceca, es decir, $P(H) = P(T) = 0.5$.

(H)²⁰. Los posibles eventos ω del espacio de eventos Ω son $\Omega = \{TT, TH, HT, HH\}$, es decir, que salga ceca y ceca, ceca y cara, cara y ceca, y cara y cara. Uno podría pensar que la variable aleatoria podría tomar un 1 para cara y un 0 para ceca, pero recordemos la definición de la variable X . Lo que nos interesa es la cantidad de caras y es por eso que, en este caso, la variable aleatoria X va a tomar los valores $X = \{0, 1, 1, 2\}$, correspondientes a cada evento aleatorio $\omega \in \Omega$. Esto, en resumidas cuentas, es lo que hace una variable aleatoria, convertir un evento aleatorio (por ejemplo, HH) en un valor numérico (en este caso 2).

En el ejemplo anterior X es lo que se denomina una variable aleatoria discreta, es decir, que sólo puede tomar algunos valores posibles (en este caso $X = \{0, 1, 1, 2\}$). Pero es importante recordar que también existen variables aleatorias continuas como por ejemplo la altura de una nueva persona que nace o la temperatura de Buenos Aires en un día de enero .

Una pregunta que nos podemos hacer es: ¿para que nos interesa definir una variable aleatoria si lo que queremos estudiar son los eventos aleatorios? Bueno, porque traducir esos eventos (que pueden ser dos tiradas de monedas o algo mucho más complejo, como tener o no tener una enfermedad) en un número tiene grandes ventajas al momento de modelizar la distribución de probabilidades.

2.3 Probabilidad

El concepto de probabilidad es algo complejo, pero, como esto no es un curso de probabilidad, vamos a confiar en que ustedes ya lo tienen claro. Si tienen coraje pueden ir a leer el capítulo 1 de (Wasserman 2004) y, si quieren algo más terrenal, pueden ir a ver el repaso de probabilidad de (Cunningham 2021) (disponible online). Pero, de forma general, podemos decir que la probabilidad es una medida numérica de la incertidumbre -o la certeza- de que un evento aleatorio ocurra²¹.

Como dijimos que tenemos una moneda justa, es razonable pensar que todos los eventos aleatorios en este espacio de eventos $\Omega = TT, TH, HT, HH$ son equiprobables, es decir, que cada

²⁰ Cara será **H**, del inglés *Head* y ceca será **T**, del inglés *Tail*.

Wasserman, Larry. 2004. *All of statistics: a concise course in statistical inference*. Springer Science & Business Media.

Cunningham, Scott. 2021. *Causal inference: The mixtape*. Yale university press.

²¹ Existen dos grandes interpretaciones de la probabilidad: la frecuentista y la bayesiana. La primera define a la probabilidad como la frecuencia relativa de un evento en una serie de experimentos repetidos, mientras que la segunda la define como una medida subjetiva de la creencia o confianza en que un evento ocurra. Si quieren bucear en las profundas aguas de este debate pueden empezar por [acá](#).

uno de ellos tiene la misma probabilidad de ocurrir, ¿no? Entonces, la probabilidad de ocurrencia de cada evento será de $1/4$ ²². ¿Pero podemos decir lo mismo de los posibles valores de $\mathbf{X}$? ¿Tiene la misma probabilidad de valer 0 que de valer 1?

Como confío que son gente inteligente, y un poco ansiosa, ya deben haber chusmeado esta tabla y se habrán dado cuenta que la respuesta es no.

ω	$X(\omega)$	$P(\omega)$
TT	0	1/4
TH	1	1/4
HT	1	1/4
HH	2	1/4

Lo que vemos es que hay dos eventos aleatorios (TH y HT) que se “mapean” al mismo valor de $\mathbf{X}$ (en este caso 1) y esto hace que la probabilidad de que $\mathbf{X}$ tome el valor de 1 no sea de $1/4$ sino de $1/2$.

Este bello y simplísimo ejemplo tiene el único objetivo de pensar un poco para qué sirve una variable aleatoria y cómo es diferente de un evento aleatorio. No vamos a entrar en más detalle.

Cuando las variables aleatorias son continuas la cosa se complica un poco más ya que $P(X = c)$ (por ejemplo, la probabilidad de que una variable tome un valor dado) es **cero**. Esto lo vamos a repensar un poco en la siguiente sección, cuando definamos lo que nos importa para este libro: las funciones de densidad y de distribución.

2.4 Probabilidad condicional

La probabilidad condicional es la probabilidad de que ocurra un evento A dado que ocurrió un evento B y se escribe como $P(A|B)$. Esta puede calcularse como:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (2.1)$$

²² El enfoque frecuentista de la estadística justamente se basa en controlar los errores dada una repetición infinita de mi experimento.

donde $P(A \cap B)$ es la probabilidad de que A y B ocurran juntos. Esta fórmula se puede entender fácilmente a partir de la definición de probabilidad como frecuencia relativa. Si tenemos una serie de experimentos repetidos, la probabilidad de que ocurra A dado que ocurrió B es simplemente la proporción de veces que ambos eventos ocurrieron juntos ($P(AB)$) sobre la proporción de veces que ocurrió B ($P(B)$).

Pensemos en el ejemplo de tirar dos monedas justas. Supongamos que queremos saber la probabilidad de que salga exactamente una cara (A) dado que salió al menos una cara (B). En este caso, $P(A \cap B)$ es la probabilidad de que salga exactamente una cara y al menos una cara, lo cual es simplemente la probabilidad de que salga exactamente una cara ($P(X = 1)$), es decir, $P(A \cap B) = P(HT) + P(TH) = 1/2$. Por otro lado, $P(B)$ es la probabilidad de que salga al menos una cara ($P(X = 1) + P(X = 2)$), lo cual es igual a $P(B) = P(HT) + P(TH) + P(HH) = 3/4$. Entonces, al reemplazar en la fórmula tenemos que:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{1/2}{3/4} = \frac{2}{3} \quad (2.2)$$

Por ahora quedémonos con esta definición simple que será de vital importancia para lo que sigue.

2.5 Probabilidad total

Imaginemos que queremos calcular la probabilidad de un evento B a partir de su relación condicional con A . Para esto, debemos considerar la probabilidad de que ocurra B tanto si A ocurre como si no. Al ponderar cada uno de estos escenarios por la probabilidad de que ocurran (o no), obtenemos lo que se denomina probabilidad total, la cual se expresa de la siguiente forma²³:

$$P(B) = P(B|A) \times P(A) + P(B|\neg A) \times P(\neg A) \quad (2.3)$$

Pensemos un ejemplo. Imaginemos que una persona tiene que completar un trabajo de jardinería (evento B) en un día. La

²³ Recordemos que $\neg$ es el símbolo lógico de la negación.